

Q1. (a) (6 marks)

(i) $\Omega(n^2)$	(ii) $\Omega(2^{1.001n})$	(iii) $\Omega(n!)$
(iv) $\Omega(n^2)$	(v) $\Omega(\log n)$	(vi) $\Omega(\log^{32} n)$

(b)

$\exists c=1, n_0 = 2^5 \bullet \forall n > n_0 \bullet$
 $n \geq 2^5$
 $n \geq (2/c)^5$
 $n^{0.2} \geq 2/c$
 $c n^{0.2} \geq 2$
 $2n \leq c n^{1.2}$

(6 marks)

(c)

Equivalent to prove $\forall c, n_0 \bullet \exists n > n_0 \bullet 2n < c n^{1.2}$
 Proof: $\forall c, n_0 \bullet \exists n = \max(\lceil (2/c)^5 \rceil, n_0) + 1 > n_0 \bullet$
 $n > (2/c)^5$
 $n^{0.2} > 2/c$
 $c n^{0.2} > 2$
 $2n < c n^{1.2}$

(6 marks)

(d) (7 marks)

To prove $O(n) \subset O(2n)$
 $\forall g(n) \in O(n), \exists c_0, n_0 \bullet \forall n > n_0 \bullet g(n) \leq c_0 n$
 Then, $\exists c_1 = c_0/2, n_1 = n_0 \bullet \forall n > n_1 \bullet g(n) \leq c_1 2n$
 Then, $g(n) \in O(2n)$.

 To prove $O(2n) \subset O(n)$
 $\forall g(n) \in O(2n), \exists c_0, n_0 \bullet \forall n > n_0 \bullet g(n) \leq c_0 2n$
 Then, $\exists c_1 = 2c_0, n_1 = n_0 \bullet \forall n > n_1 \bullet g(n) \leq c_1 n$
 Then, $g(n) \in O(n)$.

[Total : 25 marks]

Q2. (a) (i)

$a = 5, b=5, f(n) = n \log n$
 $\exists k=1 \bullet f(n) = n \log n = n^{\lceil \log_5 5 \rceil} \log n = n^{\lceil \log_b a \rceil} \log^k n$
 $f(n) = \Theta(n^{\lceil \log_b a \rceil} \log^k n)$
 Case 2 is applied.
 $T(n) = \Theta(n^{\lceil \log_b a \rceil} \log^{k+1} n)$
 $T(n) = \Theta(n \log^2 n)$

(4 marks)

(ii)

$a = 5, b=5, f(n) = n^{0.9} \log n$
 $\exists \epsilon=0.05 \bullet f(n) = n^{0.9} \log n < n^{0.95} = n^{\lceil \log_5 5 - 0.05 \rceil} = n^{\lceil \log_b a - \epsilon \rceil}$
 $f(n) = O(n^{\lceil \log_b a - \epsilon \rceil})$
 Case 1 is applied.
 $T(n) = \Theta(n^{\lceil \log_b a \rceil})$
 $T(n) = \Theta(n)$

(4 marks)

(iii)

$$a = 5, b=5, f(n) = n^2 \log n$$

$$\exists \epsilon = 1 \bullet$$

$$f(n) = n^2 \log n > n^2 = n^{\lceil \log_5 5 + 1 \rceil} = n^{\lceil \log_b a + \epsilon \rceil}$$

$$f(n) = \Omega(n^{\lceil \log_b a + \epsilon \rceil})$$

Regularity checking

$$\exists c = 1/5 < 1 \bullet$$

$$af(n/b) = 5f(n/5) = 5(n/5)^2 \log(n/5) = (n^2/5) \log(n/5) < (1/5)n^2 \log n = cn^2 \log n = cf(n)$$

Case 3 is applied.

$$T(n) = \Theta(f(n))$$

$$T(n) = \Theta(n^2 \log n)$$

(4 marks)

(b)

(i)

$$\begin{aligned} T(n) &= 5T(n/5) + n \\ &= 5[5T(n/5^2) + n/5] + n \\ &= 5^2 T(n/5^2) + 2n \\ &= 5^2 [5T(n/5^3) + n/5^2] + 2n \\ &= 5^3 T(n/5^3) + 3n \\ &= 5^k T(n/5^k) + kn \\ &= nT(1) + kn, \quad \text{set } 5^k = n, k = \log_5 n \\ &= n + n \log_5 n \\ &= \Theta(n \log n) \end{aligned}$$

(4 marks)

(ii)

Claim: $T(n) < c n \log_5 n, c > 1, n > 5$

$$\begin{aligned} T(n) &= 5T(n/5) + n \\ &< 5 c(n/5) \log_5(n/5) + n \\ &= cn (\log_5 n - 1) + n \\ &= cn \log_5 n - cn + n \\ &< cn \log_5 n, \text{ since } c > 1. \end{aligned}$$

Ans: $T(n) = O(n \log n)$

(4 marks)

(c)

$T(n) = T(nL) + T(nR) + 1$, where $n = nL + nR + 1$, and all variables are positive integers.

Claim: $T(n) < cn, c > 1$.

$$\begin{aligned} T(n) &= T(nL) + T(nR) + 1 \\ &< c nL + c nR + 1 \\ &< c nL + c nR + c, \text{ since } c > 1 \\ &= c n \end{aligned}$$

We cannot use master method, because the format of $T(n)$ does not match.

We cannot use expansion, because there will be many terms which cannot be simplified and merged.

(5 marks)

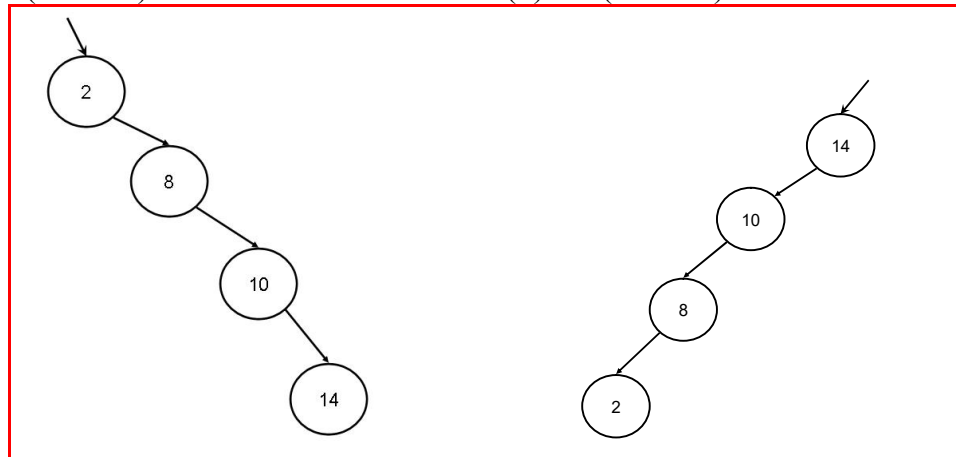
[Total : 25 marks]

Q3. (a) (8 marks)

```
// pre-condition: ptr is not NULL
Algorithm distance(ptr)
If ptr->parent =  $\emptyset$ 
    ptr->distance = 0
else
    ptr->distance = 1 + ptr->parent->distance;

if ptr->left  $\neq \emptyset$ 
    call distance(ptr->left);
if ptr->right  $\neq \emptyset$ 
    call distance(ptr->right);
return;
```

(b) (i) (2 marks)



(ii) (2 marks)

(iii) (8 marks)

```
Declare marked[0..n-1]; // n = #node
Initialize marked[0..n-1] to 0.
// marked[i]=1 means distance i has been outputted.
// pre-condition: ptr is not NULL
Algorithm distance(ptr)
If ptr->parent =  $\emptyset$ 
    ptr->distance = 0
    output ptr->element;
    marked[0] = 1;
else
    ptr->distance = 1 + ptr->parent->distance;
    if marked[ptr->distance] = 0
        output ptr->element;
        marked[ptr->distance] = 1;
if ptr->left  $\neq \emptyset$ 
    call distance(ptr->left);
if ptr->right  $\neq \emptyset$ 
    call distance(ptr->right);
return;
```

(iv) (5 marks)

```
// pre-condition: the binary tree is full
Algorithm distance(ptr)
If ptr =  $\emptyset$ 
    Return;
output ptr->element;
call distance(ptr->left);
Return;

Time complexity =  $O(\log n)$ , because the tree height
is  $\log(n+1)$ .
```

[Total : 25 marks]

Q4. (a) (12 marks)

Input: n integers.

1. If n is a power of 2 // the binary format has a single bit equal to one
 - 1.1 Execute SORT() with the n integers, and get the output.
 - 1.2 Return the output.
2. If n is not a power of 2 // more than one bit equal to one.
 - 2.1 find m such that $2^{m-1} < n < 2^m$.
 - 2.2 search for the maximum integer X in the input
 - 2.3 Execute SORT() with the n integers, plus $2^m - n$ repetitions of X , and get the output sequence S .
 - 2.4 Remove X for $2^m - n$ times from S .
 - 2.5 Return S .

Time complexity analysis

Step 1, $\log n$

Step 1.1, $n \log n$

Step 2, $\log n$

Step 2.1, $\log n$

Step 2.3, $2^m \log 2^m = O(n \log n)$

Step 2.4, n

Summing up, time complexity is $O(n \log n)$

(b)

(i) (5 marks)

```
Algorithm cost(i,n)
If i=0 or n=0
    Return 0;
If  $w_i > n$ 
    Return cost(i-1,n);
Return  $\max(w_i + \text{cost}(i-1, n - w_i), \text{cost}(i-1, n))$ ;
```

(ii) (8 marks)

```
Define C[0..p, 0..N];
For n = 0 to N
    C[0,n]=0;
For i = 0 to p
    C[i,0]=0;
For i = 1 to p
    For n = 1 to N
        If  $w_i > n$ 
            C[i,n] = C[i-1,n];
        Else
            C[i,n] =  $\max(w_i + C[i-1, n - w_i], C[i-1, n])$ ;
Return C[p,N];
```

[Total : 25 marks]

Q5. (a) (4 marks)

The adjacency list of a vertex is a linked list which nodes are the neighbours of the vertex.

An array of pointers is used, and each cell represents a vertex in G , and its pointer is pointing to the adjacency list of the vertex.

Then an edge outgoing from the vertex in G is represented by a node in the adjacency list of the vertex.

(b) (4 marks)

$O(n + e)$, or $O(e)$ for connected graphs.

There are n pointers in the array. Each node in an adjacency list represents an outgoing edge. There are $2e$ outgoing edges in G . Hence, $O(n + e)$.

(c) (6 marks)

The zeroth row stores the number of remaining rows.

The first row stores the adjacency list of vertex 1, and so on so forth.

A blank line means the adjacency list is empty.

Algorithm get_input(text file F)

Read the number of vertices, N , from the first line of F .

Create an array $A[1..N]$ of pointers.

For $i = 1$ to N

 Read a line of integers and create a linked list in order.

$A[i]$ points to the linked list.

 If the line in F is blank, then $A[i]$ is NULL.

(d) (i) (4 marks)

From the algorithm we know the following points:

(1) only grey vertices can stay in the queue. Every iteration will take exactly one vertex out of the queue.

(2) each vertex can go through the change of its colour from initially-white to grey, and from grey to black, and once it is turned black, it will stay black forever, and all colour changes are irreversible.

Hence, every vertex can only turn to grey only once.

Hence, the number of vertices that will be taken out from the queue is bounded by n .

Hence, the maximum #iteration is n .

(ii) (4 marks)

$2e$

The vertex is enqueued into Q only once, it is processed in the while loop only once, and lastly, it is taken away from Q .

Since each vertex was initially white, and was then turned grey, and lastly from grey to black, it is processed in the while loop only once, and its adjacency list will be traversed once in the forloop. Summing up the length of all adjacency lists, it is $2e$, and it is the total number of iterations of the for loop.

(iii) (3 marks)

We choose the lines “ $u \leftarrow \text{head}[Q]$ ” and “if color[u] = white” for analyzing the time complexity.

The number of executions for line “ $u \leftarrow \text{head}[Q]$ ” is the number of iterations of the while loop, which is bounded by n .

The number of executions for line “if color[u] = white” is the total number of iterations of the forloop (summing up for all iterations of the while loop), which is $2e$.

Hence, time complexity is $O(n+e)$.

[Total : 25 marks]